SQL Interview Question Bank

Query writing, differences & NULL handling · CrackAnalytics · by Mahendra Singh

Explore more — click on this link: https://medium.com/@mahendraa1188

Query-writing questions

Q1. Write a query to fetch the second-highest salary from an employee table [Easy]

Option 1: Using LIMIT + OFFSET (MySQL/PostgreSQL)

```
SELECT DISTINCT salary
FROM employee
ORDER BY salary DESC
LIMIT 1 OFFSET 1;
```

What's happening here?

- `DISTINCT salary`: ensures we don't get duplicates.
- `ORDER BY salary DESC`: ranks the salaries from highest to lowest.
- `LIMIT 1 OFFSET 1`: skips the first result (the highest) and returns the next one (the second highest).

Option 2: Using a Subquery (works in all SQL flavors)

```
SELECT MAX(salary)
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

How it works: the subquery fetches the highest salary; the outer query finds the maximum salary less than the highest — giving the second highest.

Bonus tip — what if you need the 3rd, 4th, or nth highest salary? Use `OFFSET n−1`, or a window function like `DENSE_RANK()`.

Q2. Write a query to find employees earning more than their managers [Medium]

Assume the table `employees` has: emp_id, name, salary, manager_id

```
SELECT e.name AS employee, e.salary,
       m.name AS manager, m.salary AS manager_salary
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

- **Self-join:** matches employees (e) with their managers (m).
- Filters those where employee's salary > manager's salary.

Show the difference in salary:

```
SELECT e.name, e.salary - m.salary AS salary_difference
FROM employees e
JOIN employees m ON e.manager_id = m.emp_id
WHERE e.salary > m.salary;
```

Q3. Interviewer: Find the Nth highest salary (e.g. 3rd highest) — scenario [Medium]

You are given a table Employee with columns id, name, and salary. Write a query to find the 3rd highest salary.

Approach 1: Using LIMIT with OFFSET

We can sort the salaries in descending order and then use the OFFSET clause to skip the top 2 salaries and fetch the next one (which will be the 3rd highest).

```
SELECT DISTINCT salary
FROM Employee
ORDER BY salary DESC
LIMIT 1 OFFSET 2;
```

- `ORDER BY salary DESC`: sorts the salaries in descending order.
- `LIMIT 1 OFFSET 2`: skips the top 2 salaries and fetches the next one (3rd highest).

Approach 2: Using a Subquery

Alternatively, we can use a subquery that returns distinct salaries and find the 3rd highest by comparing with the maximum.

```
SELECT MAX(salary)
FROM Employee
WHERE salary < (SELECT MAX(salary)
                FROM Employee
                WHERE salary < (SELECT MAX(salary)
                                FROM Employee));
```

- The innermost subquery finds the highest salary.
- The middle subquery finds the second-highest salary.
- The outer query returns the 3rd highest salary.

Q4. Interviewer: What is the difference between WHERE and HAVING? [Easy]

1. WHERE Clause:

- The WHERE clause is used to filter rows *before* any grouping is done.
- It applies conditions to individual rows in a table.
- If you want to filter data based on a column's value, you use WHERE.

Example: find all orders where the amount is greater than 100:

```
SELECT * FROM orders
WHERE amount > 100;
```

2. HAVING Clause:

- The HAVING clause is used to filter groups *after* the GROUP BY operation.
- It applies conditions to groups of rows that result from the grouping.
- If you want to filter based on an aggregated value (like the sum, average, etc.), you use HAVING.

Example: find customers who have made total purchases over 500:

```
SELECT customer_id, SUM(amount) AS total_amount
FROM orders
GROUP BY customer_id
HAVING SUM(amount) > 500;
```

Q5. Interviewer: How would you improve the performance of queries involving joins on large tables? [Hard]

To improve the performance of queries involving joins on large tables, there are several key techniques you can use:

- **Partitioning:** Ensure that the large tables are partitioned based on the join key. Partitioning helps reduce the amount of data processed, as only relevant partitions are scanned.
- **Bucketing:** In addition to partitioning, you can bucket the tables on the join columns. This distributes data more evenly across the buckets and reduces shuffling during the join process.
- **Map-Side Join:** If one of the tables is small enough, you can use a map-side join (also called broadcast join). This sends the small table to all the nodes, avoiding a shuffle and speeding up the join.
- **Optimize Join Types:** Use the appropriate join type based on the data size — broadcast join for small tables, sort-merge join for larger sorted datasets, bucketed map join if both tables are bucketed on the join key.
- **Increase Parallelism:** Adjust the number of reducers or partitions in Spark/Hive to better distribute the join processing workload across available resources.
- **Use EXPLAIN:** Before running a query, use the EXPLAIN command to understand how the join is being executed and identify bottlenecks in the query plan.

Q6. Interviewer: How would you optimize a slow-running SQL query? [Hard]

- **Check Indexes:** Ensure that the columns used in WHERE, JOIN, and ORDER BY clauses have appropriate indexes.
- **Analyze Query Execution Plan:** Use tools like EXPLAIN to see how the query is executed and identify bottlenecks.
- **Optimize Joins:** Use appropriate join types (INNER JOIN, LEFT JOIN, etc.) and reduce the number of joins if possible.
- **Filter Early:** Apply filters in the WHERE clause as early as possible to reduce the amount of data processed.
- **Limit Data:** Select only the columns you need instead of using SELECT * and use LIMIT to restrict the number of rows.
- **Use Caching:** Cache frequently accessed data to avoid repeated computations.

These steps should help improve the query's performance.

Must-know differences

Q1. RANK vs DENSE_RANK *[Medium]*

- **RANK:** Provides a ranking with gaps if there are ties.
- **DENSE_RANK:** Provides a ranking without gaps, even in the case of ties.

For salaries 100, 100, 90 → RANK gives 1, 1, 3 while DENSE_RANK gives 1, 1, 2.

Q2. HAVING vs WHERE clause *[Easy]*

- **WHERE:** Filters rows before grouping.
- **HAVING:** Filters groups after the GROUP BY clause.

Q3. UNION vs UNION ALL *[Easy]*

- **UNION:** Removes duplicates and combines results.
- **UNION ALL:** Combines results without removing duplicates (faster).

Q4. JOIN vs UNION *[Easy]*

- **JOIN:** Combines columns from multiple tables.
- **UNION:** Combines rows from multiple tables with similar structure.

Q5. DELETE vs DROP vs TRUNCATE *[Easy]*

- **DELETE:** Removes rows, with the option to filter (WHERE).
- **DROP:** Removes the entire table or database.
- **TRUNCATE:** Deletes all rows but keeps the table structure.

Q6. CTE vs TEMP TABLE *[Medium]*

- **CTE:** Temporary result set used within a single query.
- **TEMP TABLE:** Physical temporary table that persists for the session.

Q7. SUBQUERIES vs CTE *[Medium]*

- **Subqueries:** Nested queries inside the main query.
- **CTE:** Can be more readable and used multiple times in a query.

Q8. ISNULL vs COALESCE *[Medium]*

- **ISNULL:** Replaces NULL with a specified value, accepts two parameters.
- **COALESCE:** Returns the first non-NULL value from a list of expressions, accepting multiple parameters.

Q9. INTERSECT vs INNER JOIN *[Hard]*

- **INTERSECT:** Returns common rows from two queries.
- **INNER JOIN:** Combines matching rows from two tables based on a condition.

Q10. EXCEPT vs NOT IN *[Hard]*

- **EXCEPT:** Returns rows in the first query but not in the second.
- **NOT IN:** Filters rows where a column's value is not in a given list.

NULL questions

1. What is NULL in SQL? *[Easy]*

NULL represents the absence of a value in a field. It is not the same as an empty string or zero; it is an unknown or undefined value.

2. How to check for NULL values in a column? *[Easy]*

Use the `IS NULL` or `IS NOT NULL` condition in the WHERE clause.

```
SELECT * FROM tableName WHERE columnName IS NULL;
```

3. What is the difference between NULL and an empty string? *[Easy]*

NULL represents the absence of a value, while an empty string is a valid string with zero length.

4. How to replace NULL values with a specific value in a query result? [Easy]

Use the `COALESCE` function.

```
SELECT COALESCE(columnName, 'Replacement Value') AS columnName
FROM tableName;
```

5. Explain the behaviour of NULL in aggregate functions [Medium]

If an aggregate function encounters a NULL value, it generally ignores it — except for the `COUNT(*)` function, which counts all rows, including those with NULL values.

6. Can a table have multiple NULL values in a unique key column? [Medium]

In SQL Server, you can have only one NULL value in a unique key column.

7. How to insert a NULL value into a column during data insertion? [Easy]

Simply omit the column from the INSERT statement or explicitly use the keyword NULL.

```
INSERT INTO tableName (column1, column2) VALUES (value1, NULL);
```

8. Explain the use of the ISNULL function [Medium]

The ISNULL function returns the specified replacement value if the expression is NULL; otherwise, it returns the expression itself.

```
SELECT ISNULL(columnName, 'Replacement Value') AS columnName
FROM tableName;
```

9. How to count the number of NULL values in a column? [Medium]

Use the COUNT function with a CASE statement.

```
SELECT COUNT(CASE WHEN columnName IS NULL THEN 1 END) AS NullCount
FROM tableName;
```

10. Can NULL values be indexed in SQL Server? [Hard]

Yes, NULL values can be indexed. However, keep in mind that querying for NULL values might be less efficient than querying for non-NULL values due to the way indexes work.